2018 IEEE 38th International Conference on Distributed Computing Systems

Continuous and Parallel LiDAR Point-cloud Clustering

Hannaneh Najdataei, Yiannis Nikolakopoulos, Vincenzo Gulisano, Marina Papatriantafilou
Chalmers University of Technology, Sweden - {hannajd,ioaniko,vinmas,ptrianta}@chalmers.se

***Abstract*—In distributed digitalized environments in the context of the Internet of Things, we often need to do an analysis of big data originating at high rate-sensors at the edge of the infrastructure. A characteristic example is the light detection and ranging (LiDAR) technology, that allows sensing surrounding objects with fine-grained resolution in large areas. Their data (known as point clouds), generated continuously at very high rates, through appropriate analysis can provide information to support automated functionality in distributed cyber-physical systems; clustering of point clouds is a key problem to extract this type of information. Methods for solving the problem in a continuous fashion can facilitate improved processing in fog architectures, through enabling low-latency, efficient continuous and streaming processing of data close to the sources; moreover, parallelism is a key requirement to exploit a variety of computing architectures in this context.**

We propose *Lisco*, a single-pass continuous Euclidean-distance-based clustering of LiDAR point clouds, that maximizes the granularity of the data processing pipeline and thus shows the potential for data- and pipeline-parallelism. We further present its parallel version, *P-Lisco*, that is architecture-independent and exploits the parallelism revealed by *Lisco*'s algorithmic approach. Besides their algorithmic analysis, we provide a thorough experimental evaluation on architectures representative of high-end servers and of resource-constrained embedded devices and highlight the multiplicative improvements and scalability benefits of the proposed algorithms compared to the baseline, using both real-world datasets as well as synthetic ones to fully explore a wide spectrum of stress-levels for the algorithms.

***Keywords*-streaming; parallel; clustering; point-cloud; LiDAR; edge computing; fog computing; data analysis**

I. Introduction

Sensors that take measurements of systems' environments and generate big data in form of large streams of readings are increasingly common in distributed, cyber-physical systems, in the context of Internet-of-Things. In these contexts, data analysis can benefit from being carried out at the edge of the distributed infrastructures, for reasons of efficiency in latency, energy, communication-bandwidth. The LiDAR (light detection and ranging) sensor is a prominent example. A LiDAR commonly mounts several lasers on a rotating column; at each rotation step, these lasers shoot light rays and, based on the time the reflected rays take to reach back the sensor, produce a stream of distance readings at high rates, in the realm of million of readings per second.

Efficient analysis and distilling of valuable information from high-rate raw measurements is challenging [5], [8], [22], [36]. A common way of extracting valuable information from LiDAR sensor data is *clustering* of its raw distance measurements in order to detect objects surrounding the sensor [27]. This can enable the detection of surrounding obstacles and prevent accidents or study the motion feasibility of objects in factories' production paths [1], [37].

Challenges and contributions

The processing time incurred by clustering of raw LiDAR measurements (aka *point-clouds*) is one of the main challenges in this context because of the high rates and the need for the clustering outcome to be available in a timely manner to be useful. Point-clouds can also be generated through combining points from camera images, to obtain a 3D perception of the environment. In cyber-physical setups, where analysis can be pushed to the edge to run in a distributed and parallel fashion (e.g., to have smart vehicles analyzing locally their own surroundings), these challenges are exacerbated by the reduced computational power of edge embedded devices. Finally, the accuracy of the clustering is challenging as well, since readings from objects that are at different distances from the sensor can vary a lot in density.

A key performance enabler for high-rate data streaming analysis is the joint pipelining and parallelization of its composing tasks. Nevertheless, common state-of-the art approaches for clustering of LiDAR data (cf [34], [26], [31] and more detail in § VII elaborating on related work) first organize the points to be clustered (e.g. sorting them so that points close in space are also close in the data structure maintaining them) and only then perform the clustering by querying the organized data (e.g. by running neighbor-query as discussed in § II). By doing this, they introduce a batch-based analysis that affects the clustering performance.

To overcome this, we target single-pass analysis, that can enable fine-grained pipelining and parallelism. We first propose a new method for point-cloud clustering, that allows to boost processing throughput by maximizing the internal pipelining of the data analysis steps, through a key idea that exploits the ordering of the data generated by the sensor. Subsequently, we introduce its parallel counterpart, which is architecture-independent and scales the analysis by exploiting the inherent disjoint-access parallelism of the algorithm's pipeline and of the calculations needed for each point. In more detail, we make the following contributions: 1) We introduce the algorithms *Lisco* and *P-Lisco*, which provide Euclidean-distance-based clustering of LiDAR point-clouds, in a way that exploits the granularity of the data processing pipeline and its parallelism without the need for supporting sorting data structures.

2575-8411/18/$31.00 ©2018 IEEE
DOI 10.1109/ICDCS.2018.00071

2) We provide a fully implemented prototype of *Lisco* and *P-Lisco* and show their equivalence and complexity improvements in connection to the state-of-the-art Euclidean-distance-based clustering method in the Point Cloud Library (PCL)[1], which we adopt as baseline due to its known accuracy, efficiency and wide use-base [17], [27], [30] in applications for environment perception (e.g. object detection, tracking, localization and mapping) in automated systems.
3) We analyze the complexity behaviour of the algorithms and we present a thorough comparative evaluation, using both real-world and synthetic data, to explore in detail the spectrum of trade-offs. We use hardware representative of both high-end servers and resource-constrained devices, embracing a spectrum of distributed edge/fog infrastructures, from production environments to automated vehicles/robots. We show a significant improvement, up to 10 times better latency than the baseline sequentially, and we also show the parallelization benefits. The improvements are more pronounced in the resource-constrained systems. Based on these, we can observe that *Lisco* offers high benefits already in its sequential form and in the cases where even faster processing is required, the parallel version *P-Lisco* is useful.

In the rest of the paper, § II overviews the LiDAR sensor, the data it produces and clustering-related techniques that exist for such data. § III explains the main idea and challenges of continuous clustering. § IV presents the algorithmic implementation and analysis of *Lisco* while the corresponding discussion for *P-Lisco* is in § V . We evaluate the proposed methods in § VI. Finally, we discuss related work and conclude in § VII and § VIII, respectively.

II. Preliminaries

In this section, we describe key properties of LiDAR sensors and their data. We also provide a detailed problem description and evaluation criteria of solutions. Finally, we outline the Euclidean-distance based clustering in PCL, which we use as baseline as explained in the introduction.

A. LiDAR - sensor and data

The LiDAR sensor mounts L lasers in a column, each measuring the distance from the target by means of the time difference from emitted and reflected light pulses. The column of lasers performs R rotations per second, each consisting of S *steps*, producing a set of n points, also called *point-cloud*. The number of points reported by the LiDAR sensor for each rotation can be lower than $L \times S$, since some of the emitted pulses might not hit any obstacle. We refer to the angle in the x-y plane between two consecutive steps[2] as $\Delta\alpha$ (e.g. measured in *radians*) and to the elevation angle from the x-y plane between two consecutive lasers as $\Delta\theta$.

[1]Available through: `http://pointclouds.org`

[2]If the horizontal angle between steps is not constant and lasers are not perfectly aligned in the x-y plane, then $\Delta\alpha$ refers to the minimum such angle. Similarly, if the vertical angle between lasers is not constant, $\Delta\theta$ refers to the minimum such angle.

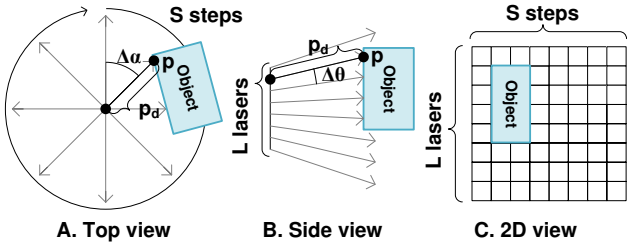


Figure 1: *Top and side views for the LiDAR's light pulses showing steps and lasers, and its resulting 2D view. In the 2D view, non-reflected pulses are white while reflected ones are coloured.*

Each point p is described through attributes $\langle d, l, s\rangle$ where $d \geq 0$, l, s are the measured *distance*, the *laser index* and the *step index*. Value 0 for d shows no reflection in the point. In the following, we use the notation p_x to refer to attribute x of point p (i.e., p_d refers to the distance reported for point p).

We visualize tthe points by unfolding them as a 2D matrix M, where each column contains L rows and each step is a column. We assume that new data is delivered from the physical sensor with the granularity of one *step*. Figure 1 shows an example with $L = 8$ and $S = 8$.

B. Problem formulation: from point-cloud to clusters

Given a set of points corresponding to LiDAR measurements, we want to identify disjoint groups of them that can be potential objects in the environment surrounding the sensor. A natural criterion, commonly used in the literature and applications, is the distance between points. In particular, adopting the problem definition in [25] (Chapter 4), which we paraphrase here for ease of reference:

Definition 1. [*Euclidean-distance based clustering*] Given n points in 3D space, we seek to partition them into some (unknown) number of clusters C, such that every cluster contains at least a predefined number of points ($minPts$), that is $\forall j, |C_j| \geq minPts$, and all clusters are disjoint, that is $C_i \cap C_j = \emptyset, \forall i \neq j$; points p_i and p_j should be clustered together if $||p_i - p_j||_2 \leq \epsilon$, with ϵ being a predefined threshold.

To facilitate the presentation of the baseline algorithm and our proposed ones, we introduce the *ϵ-neighborhood of a point p*: the set of input points whose Euclidean distance from p is at most ϵ. A set of points closed under the union of their ϵ-neighborhoods is characterized as *noise* if its cardinality is less than $minPts$.

Note that, when clustering data from scenarios like the vehicular ones, a pre-processing task is usually defined to filter out points that refer to the ground, since many objects lying on it would be otherwise clustered together. Since this can be implemented as a non-expensive and continuous filtering operation [11] (e.g. by removing points below a certain threshold, as we do in § VI), we do not further discuss it in the remainder.

C. Euclidean-distance-based clustering in PCL

The Point Cloud Library (PCL) [27] provides a collection of state-of-the-art algorithms implemented to process 3D data, including filtering, clustering, surface reconstruction and more. In this section we review its method for cluster extraction, which is an Euclidean-distance based clustering and we use it as a baseline. For brevity we call this algorithm *PCL_E_Cluster* in the rest of this document.

PCL_E_Cluster works on batches of data points. It first builds a k-dimensional tree (aka kd-tree) to facilitate finding the nearest neighbors of points [27], similarly as the DB-SCAN algorithm [4]. Subsequently, it proceeds as described in Algorithm 1 to extract the clusters.

Algorithm 1 Main loop of *PCL_E_Cluster*

```
1:  clusters = ∅
2:  for p ∈ P do
3:      Q = ∅
4:      if p.status ≠ processed then
5:          Q.add(p)
6:          for q ∈ Q do
7:              q.status = 'processed'
8:              N = GetNeighbors(q, ε)
9:              Q.addAll(N)
10:         if size(Q) ≥ minPts then clusters.add(Q)
```

III. Towards Continuous Clustering

In this section, we describe the main idea behind continuous point-cloud clustering. Subsequently, we focus on the challenges and trade-offs introduced by the latter.

A. Main idea

Based on the clustering requirements in Definition 1, each point p reported by the LiDAR is grouped with each neighbor point p' within distance ϵ. A group of points is eventually delivered as a cluster if it contains at least *minPts* points, or characterized as noise otherwise. As discussed in § II-C, implementations such as the one by PCL limit the pipelining of the analysis because of processing stages that cannot be executed concurrently. More concretely, they first traverse all the points to populate a supporting data structure that facilitates finding the points in the ϵ-neighborhood of each point p (a kd-tree in the case of PCL) and subsequently traverse a second time all the points to cluster them.

As shown in Figure 2, the intuition behind continuous point-cloud clustering is that the search space for the ϵ-neighborhood of point p can be translated into a set of readings given by certain *steps* (Figure 2.A) and *lasers* (Figure 2.B) around p, and within a certain distance range from the LiDAR's emitting sensors. It can be noticed that these constraints describe a region that may also contain points whose distance from p is greater than ϵ. Nevertheless, if points within distance ϵ from p exist, they will be returned by one of these steps and lasers and will fall within the given

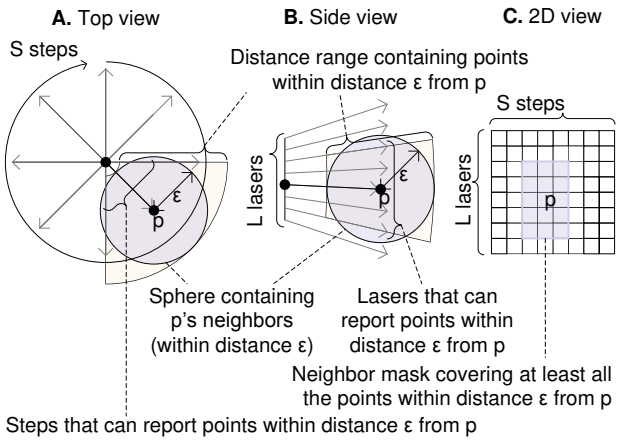


Figure 2: *Top and side views (A and B) showing which steps and lasers, respectively, to include in the neighbor mask (C) for the latter to contain at least all the points within distance ϵ from p.*

distance range, as we further explain in § IV-C. Thus, in order to discover the ϵ-neighborhood of p, by leveraging the sorted delivering of tuples from the LiDAR sensor step after step, it is enough to explore a *neighbor mask* centered at p, as shown in Figure 2.C. In this way, we eliminate the need for a search-optimized data structure (e.g. kd-trees, which may form a bottleneck, as we explain in section VII) and we can cluster the data as it is being received.

Notice that having a point p in the mask of another point p', does not mean p and p' belong to the same cluster. What we aim for with the mask is to delimit a large enough subset of points, to check their distances from p', so that we do not miss to do all the necessary comparisons that are not redundant due to transitivity.

B. Coping with the challenges of continuous clustering

Continuous clustering introduces challenges not found in batch-based approaches, as we discuss in the following.

1) Partial view of neighbor mask: As discussed in § III-A, p's neighbor mask contains all its ϵ-neighbors. Notice, though, that upon delivery of p, points referring to subsequent steps are yet to be delivered by the sensor. Nevertheless, since the neighbor masks of two points p_1 and p_2 within distance ϵ from each other will both contain both p_1 and p_2, traversing half of a neighbor mask containing the points delivered so far does not result in missing comparisons. This is a key idea in the approach proposed here.

Algorithm 2 shows how the number of previous and subsequent steps σ and the number of upper and lower lasers λ that possibly contain points within distance ϵ from p (i.e., p's neighbor mask) is computed. As discussed in § II, $\Delta\alpha$ and $\Delta\theta$ refer to the minimum angle differences between two consecutive steps and lasers, respectively.

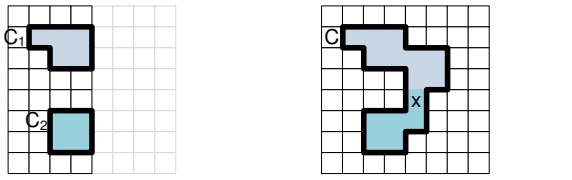
Algorithm 3 checks whether point p' is in p's neighbor mask. Take into account that, for a minority of steps, not all the points on p's left side lie on columns with a lower

Algorithm 2 Given point p, compute the number of previous steps σ and upper and lower lasers λ bounding all the points within distance ϵ from p.

```
1: procedure getNeighborMask(p)
2:     λ ← ⌈|arcsin(ϵ/p_d)|/Δθ⌉
3:     σ ← ⌈|arcsin(ϵ/p_d)|/Δα⌉
4:     return λ, σ
```

Algorithm 3 Given point p, p' and neighbor mask's boundaries check if p' is in p's neighbor mask.

```
1: procedure isInMask(p,p',λ,σ)
2:     if p'_d > 0 ∧ (0 ≤ p_s − p'_s ≤ σ ∨ 1 ≤ p'_s ≤ p_s + σ − S) ∧
       |p_l − p'_l| ≤ λ ∧ |p_d − p'_d| ≤ ϵ then
3:         return true
4:     else
5:         return false
```



A. 4 out of 8 steps have been received, 2 clusters C1 and C2 have been identified so far

B. 8 out of 8 steps have been received, 1 clusters C is eventually found

Figure 3: *Example showing how two subclusters found by Lisco's continuous analysis may eventually end up in the same cluster.*

index than p's (i.e., they are not stored on the left side of the matrix M that we described in section II-A). E.g., if a point in column 2 should be compared with 3 columns on the left ($\lambda = 3$), then it should be compared with columns S-1, S and 1. In such a case, some comparisons must be postponed until such steps are delivered. A point p' on the left side of p and within distance ϵ lies on a lower index than p if $0 \leq p_s - p'_s \leq \sigma$. On the other hand, if $p'_s + \sigma > S$, then p' is on the left side and within distance ϵ from points in steps $1, \ldots, p_s + \sigma - S$. In both cases, the clustering semantics in § II require p and p' within distance ϵ to be grouped together.

2) Continuous cluster management: A second challenge arises since *subclusters* (i.e. sets of points that have been clustered together during the processing of the so far received steps of the input) evolve as new steps arrive. A *cluster*, i.e. a set of at least *minPts* points that are clustered together, identified once the rotation is processed, could be the union of several previously discovered subclusters, as seen in Figure 3. Figure 3.A shows subclusters C_1 and C_2 found when half of the points in the rotation have been processed, while 3.B shows the ones found when all the points in the rotation have been processed. The point marked with x is the first point identified to have one neighbor in each of the two earlier disjoint subclusters. Once x is processed, these should be merged.

Consider each subcluster to have a unique identifier called *head*. Because subclusters are found continuously while the points of a rotation are being processed, the algorithm requires methods to: (1) retrieve the head of the points belonging to a subcluster and (2) merge two subclusters together in order for the final cluster to be delivered as a single item. To do this, we use in Algorithm 4 (overviewing the clustering process applied to each incoming point p) the methods `getH(Point p)` and `merge(Head H1,Head H2)`. Once two subclusters are merged invoking `merge`, we expect `getH` to return the same head for any point of the two subclusters. Without loss of generality, we assume this head to be H_1 in the following. Because of this, we define a third method `setH(point p,Head H)`. Finally, we define the method `createH()` to allow for newly discovered subclusters to be instantiated.

Algorithm 4 Given point p and p', link them (or their subclusters) together.

```
1: procedure linkage(p,p',subclusters)
2:     H1=getH(p)
3:     H2=getH(p')
4:     if H1 == ∅ ∧ H2 == ∅ then
5:         H = createH()
6:         setH(p,H)
7:         setH(p',H)
8:         subclusters.add(H)
9:     else if H1 == ∅ ∧ H2 ≠ ∅ then
10:        setH(p,H2)
11:    else if H1 ≠ ∅ ∧ H2 == ∅ then
12:        setH(p',H1)
13:    else
14:        merge(H1,H2,subclusters)
```

As shown in Algorithm 4, four cases are checked for two points within distance ϵ that are not in the same subcluster:

- *Line 4*: None of the two points belongs to a subcluster: a new subcluster head is created and set for both points.
- *Line 9 (resp. 11)*: Point p (resp. p') does not belong to a subcluster while p' (resp. p) does: then, p (resp. p') refers to the same head as point p' (resp. p).
- *Line 13*: Points p and p' belong to different subclusters: then, the two subclusters are merged.

IV. Algorithm *Lisco*

In this section, we explain details of the algorithmic implementation of *Lisco*, prove its correctness and analyze its complexity, also in comparison to *PCL_E_Cluster*.

A. Algorithmic implementation

Incoming data points are considered as entries in a matrix, M, whose number of rows and columns equal the number of lasers and steps, respectively. Upon reception of the points of a step, we store them in the corresponding column of the matrix. By using the laser and the step number, all the attributes of a point and the head of the subcluster which the point belongs to, can be extracted in constant time. The

head of a subcluster is defined as the index of the point with the lowest indices in lexicographical order of steps and lasers during the creation of a new subcluster. When two subclusters are merged, the head of the subcluster with the largest number of members is maintained.

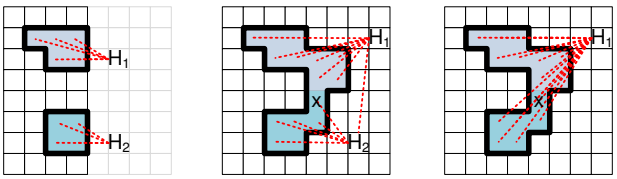
The main loop of *Lisco*, presented in Algorithm 5, processes points in M, in step and laser order (Line 4). Each point is processed only if its distance is greater than 0 (that is, if the LiDAR's pulse has been reflected for the point's step and laser index) (Line 5). Once all the points have been processed, all the clusters containing at least *minPts* points are delivered. As it can be noticed, the parameter *minPts* does not have an effect in the complexity of the algorithm, since it is only used to filter the delivered clusters at the very end of the clustering process.

Algorithm 5 Main loop of *Lisco*

```
1:  subclusters = ∅
2:  upon event reception of step s do
3:      for l ∈ 1, ..., L do
4:          p = M[l, s]
5:          if p_d > 0 then
6:              λ, σ = getNeighborMask(p)
7:              for p'|isInMask(p,p',λ,σ) do
8:                  if getH(p)≠getH(p') ∧||p − p'||_2 ≤ ε then
9:                      linkage(p,p',subclusters)
10: upon event all steps processed do
11:     for subcluster ∈ subclusters do
12:         if size(subcluster) ≥ minPts then return subcluster
```

In the algorithm, $subclusters$ is a *hash map* used in order to keep track of subclusters and their corresponding members. It is implemented as a linked list of arrays, where each key is the header of a subcluster and its members are stored in the array. If the size of a subcluster exceeds the size of the array, a new array is linked to the tail of the current array, so that subclusters can grow without restriction. At the end of the clustering procedure, we use $subclusters$ to traverse through subcluster heads. Each subcluster that has more than *minPts* members is announced as a cluster, otherwise it is characterized as noise.

The complexity of *Lisco* (analysed and compared to the complexity of *PCL_E_Cluster* in § IV-C) depends on the complexity of its methods. As shown in Algorithm 2, method `getNeighborMask` is executed in constant number of steps, since it boils down to a fixed number of numerical operations. Similarly, methods `createH` and `setH` can also induce $O(1)$ steps, as we discuss in the following. The algorithmic implementations of `getH` and `merge`, nevertheless, introduce the following trade-off. On one hand, `merge` can be implemented to induce $O(1)$ time-cost; this can be done by maintaining a hierarchy of subclusters being part of the same subcluster while incurring a higher cost for the `getH`, linear in the number of subclusters. Figure 4.B shows how some of the points clustered together once all the



A. The subclusters have different *heads*

B. After merging, one subcluster's head (H_1) points to the other (H_2)

C. After merging, all the points are directly linked to the *head*

Figure 4: *Examples in which the* `merge` *method's implementation induces* $O(1)$ *cost by hierarchically linking heads of subclusters belonging to the same subcluster (B) or, alternatively, the* `getH`*'s implementation induced cost* $O(1)$ *cost by having all points directly linked with the subcluster head (C). In (B), the complexity of* `getH` *can no longer be* $O(1)$ *but potentially linear in the number of subclusters for some of the points, while in (C), the head of all the points of a subcluster needs to be updated when the subcluster is merged with another one.*

data is processed point to head H_1 via head H_2. For these points the `getH` method has a higher cost than that of the points directly pointing to H_1, which depends on the chains induced by the data structure to maintain the hierarchy. In the proposed implementation we opt for an $O(1)$ cost for method `getH` and a higher cost for `merge`, also shown in Figure 4.C and § IV-C. The reason, given Algorithm 4 is that `getH` is executed twice for each pair of points being compared, while `merge` is executed significantly less often.

The methods in Algorithm 4 are implemented as follows:
- `createH`: This method gets two points that do not belong to any subcluster, and returns the one with the lower index-pair for step and laser. Since the returned point will be head of a subcluster, a new node is created in the $subclusters$ and the head is mapped to it.
- `setH`: This method sets the head of a point. By calling it, we are adding a point to a subcluster with an identified head. The point also needs to be added as the last element in the array of the mapped head.
- `getH`: This method reads the head point. If two points are in the same subcluster, they get the same head point as the result of this method.
- `merge`: If two points belong to different subclusters, *Lisco* merges them. It chooses the one with bigger number of members as the base, and merges the other one by changing the head of its members to the base subcluster's head. After this, it is also necessary to remove the merged subcluster's head from the $subclusters$ hash map and append its array to the base subcluster's array.

B. Correctness

Based on *Lisco*'s functionality and algorithmic implementation, we explain why its outcome satisfies Definition 1.

Claim 1. If two points p and p' are in the ϵ-neighborhood of

each other, at the end of *Lisco* they will be either in the same cluster or characterized as noise, as required in Definition 1.

Proof: (sketch) Consider that, w.l.o.g. p is processed second. Given method *getNeighborMask*, p' will be found to be in the neighbour mask of p and further, in the ϵ-neighborhood of p in the main loop of *Lisco*. This implies that they will be merged/inserted in the same subcluster. Unless that subcluster in the end is found to contain fewer points than $MinPts$, it will be return as a final cluster, after completion of the main loop of *Lisco*. ■

C. Complexity analysis

Regarding *PCL_E_Cluster*, the required processing work volume is similar to the DBSCAN algorithm, i.e. building a spatial index (kd-tree) and using it to execute region queries for each point, resulting in an overall *expected time complexity* of $O(nlogn)$ processing steps [4], [24], [32]. As we show below, this is *Lisco*'s *worst case complexity*.

Claim 2. *Lisco*'s time complexity is linear in the number of points, multiplied by a factor that depends on the size of the clusters in the set of data points. In the *worst-case* where there is a big cluster of $O(n)$ points, it can take $O(nlogn)$ processing steps for *Lisco* to complete.

Proof: (sketch) Overall, the time complexity of *Lisco* is the number of iterations in the main loop (i.e. n, as the number of points), times the work in each iteration, i.e. for each point (i) finding its ϵ-neighborhood, and (ii) linking the points in that neighborhood.

Part (i) induces an asymptotically constant cost, depending on ϵ, as it is performed through the comparisons implied by the masking operation *getNeighborMask*. The total number of comparisons for point p' to be categorized as the ϵ-neighbor of point p includes checking three conditions. The first condition is to check if p' is in the neighbor mask of p through the *isInMask* method. The second condition is if the two points belong to different subclusters. Finally, the last condition is if the Euclidean distance between p and p' is less than ϵ. These conditions are checked in steps 7 and 8 of algorithm 5.

Regarding part (ii), methods' `createH` and `setH` algorithmic implementation incurs a constant number of processing steps each, as we explain in § IV-A. Moreover, as also explained in § IV-A, `getH` induces $O(1)$ time-cost, as a point can identify its head in $O(1)$. The cost of `merge` consists of (a) a constant numbrer of steps for managing the two entries of the hash-map data structure for subclusters and (b) a set of $O(x)$ steps, where x is the size of the smaller subcluster, due to updating the head of the points of the smaller of the subclusters being merged. In the worst-case, the final clustering has a large subcluster of $O(n)$ points, and an unlikely scenario for constructing it, might require *Lisco* to merge roughly equally-sized subclusters at each of the merge operations leading to the big subcluster – any other combination of subclusters would lead at most to an equal cost as the one described. Since halving $O(n)$ points can be made at most $O(logn)$ times, the worst-case *total* number of merge-related processing steps will be dominated by a sequence of $O(nlogn)$ steps, which will be the dominating cost in the worst case complexity of *Lisco*. ■

Note that for the worst-case to happen, we would need to have a dominating object of $O(n)$ points, whose cluster of points is uncovered as described in the proof, which is a very unlikely scenario (e.g. a huge object consisting of horizontal stripes at distance $> epsilon$ from each-other, detached in the beginning of the rotation and combining gradually $O(logn)$ number of times, thus causing equivalent merges). As we observe in our evaluation, the number of merge-related operations (i.e. the resulting head-changes) which is the dominating cost in *Lisco*, is significantly lower than the worst-case asymptotic bound of $O(nlogn)$.

V. Parallel Lisco

First, we explain the main idea and the algorithmic implementation of *P-Lisco*, the parallel version of *Lisco*. We then elaborate on the properties of the parallel algorithm.

To parallelize *Lisco*, we consider the pipeline-parallelism that is revealed by its algorithmic approach, as well as the data-parallelism that it enables. We observe that two parts of *Lisco*, which we mentioned in § IV-C, can be executed in a pipeline fashion, namely the exploration of the ϵ-neighborhood of each point and the linking of the points. We name the former, where we also observe high data-parallelism, *scouting* and the second *linking* and we call the executing threads analogously. The *Scout*'s job is, given a point p, to explore its neighbor mask, find the distances between p and all the points in the mask, and return those with distance smaller than ϵ. A *Linker*'s job is to modify subclusters by linking p and the points in its ϵ-neighborhood.

A. Algorithmic implementation

We consider a system of asynchronous threads, communicating using read/write shared variables. The pseudocode for *P-Lisco*'s components *Scout* and *Linker* is in algorithms 6 and 7. Observe that *scouting* can be achieved through read-only operations on static variables of the shared data (data-parallelism) and only local updates, it is possible to parallelize among several *Scout*s, which means having several threads running the *Scout* algorithm simultaneously. The *Linker*'s job implies the need for consistent updates on the cluster data-structures. Due to the efficient algorithmic implementation of the linking functionality as explained in the previous section, we maintain its structure in the *P-Lisco* version, providing consistent information exchange with the scouting. The correctness of the outcome when parallelizing the pipeline as described here is discussed in § V-B.

It is useful to notice that the *Scout*s are producers of information (ϵ-neighborhoods of the points) that the *Linker*

needs for doing its job. To allow the *Scout*s to not be limited by the speed variations of the *Linker*, we propose that they utilize multiple buffers and we adopt a synchronization handshaking between each *Scout* and *Linker*, in order to be able to recycle the space.

The details of the *Scout* algorithm for thread i (out of T *Scout* threads) are shown in Algorithm 6. Upon receipt of a column of points, each entry (row) of M is assigned to a thread so that there is no competition in reading the points among all *Scout*s (line 3) and the workload is balanced. Each *Scout* has W work-spaces, each of which (indexed by w) can store a $list$, i.e. the ϵ-neighbors of a point, and an atomic boolean $flag_w^i$ to inform the *Linker* if the corresponding $list$ is ready to be used. In the beginning, each *Scout* initializes all its own $flag$ variables to $false$ (line 1). To store the list of neighbors of the point, the *Scout* iterates over its work-spaces to find one that it can use (that has not been used or that its contents have been consumed by the linker, i.e. one whose $flag$ is $false$), line 6. After adding all the ϵ-neighbors of the current point into this list, it changes the corresponding flag to $true$, to let the *Linker* know that the content of this list is ready to be used. Once the *Scout* has found all the ϵ-neighbors of its points, it sets its termination flag to $true$ to inform the *Linker* that no more information will be produced by it.

Algorithm 6 *P-Lisco*'s *Scout* component, using T threads. Each thread i, has W work-spaces consisting of a list and its corresponding atomic flag.

```
1:  ∀w, flag_w^i = false
2:  upon event reception of step s do
3:      for l = i; l ≤ L; l+ = T do
4:          p = M[l, s]
5:          if p_d > 0 then
6:              w ← NextAvailableWorkspace(i)      ▷ find next work-space for thread i with flag_w^i = false
7:              λ, σ = getNeighborMask(p)
8:              list_w^i[0] = p
9:              for p'|isInMask(p,p',λ,σ) do
10:                 if ||p − p'||_2 ≤ ε then
11:                     list_w^i.add(p')
12:             flag_w^i = true
13:             go to line 3
14: termination_i = true
```

As shown in Algorithm 7, the *Linker* iterates over the work-spaces of the *Scout*s, and each time it finds new information available (i.e. the ϵ-neighbors of a point), it performs $linkage$ to modify subclusters in the same way as *Lisco*. The $list$ contains all the points in the ϵ-neighborhood of the point p, even those that are already in the same subcluster as p. Therefore, to be more efficient, it is important to modify the subclusters only if the point and its neighbor do not belong to the same subcluster (line 11). After consuming the information, the *Linker* sets $flag_w^i$ to $false$ to let the corresponding *Scout* use that work-space again (line 14). Once all *Scout*s are done with their work and have set their termination flag, the *Linker* iterates one last time over the work-spaces to check if there is any unconsumed data in them. Finally, the *Linker* delivers all the clusters containing at least *minPts* points.

Algorithm 7 *P-Lisco*'s *Linker* component

```
1:  subclusters = ∅
2:  isFinished = false
3:  while !isFinished do
4:      if ∀i, termination_i == true then
5:          isFinished = true
6:      for i ∈ 1, ..., T do
7:          for w ∈ 1, ..., n do
8:              if flag_w^i then
9:                  p = list_w^i[0]
10:                 for p' ∈ list_w^i do
11:                     if getH(p)≠getH(p') then
12:                         linkage(p,p',subclusters)
13:                 list_w^i ← ∅
14:                 flag_w^i = false
15: for subcluster ∈ subclusters do
16:     if size(subcluster) ≥ minPts then return subcluster
```

B. Correctness

Based on *P-Lisco*'s functionality and algorithmic implementation, we show here why *P-Lisco*'s clustering outcome is the same as *Lisco*'s, satisfying Definition 1.

Claim 3. The synchronization between *Scout*$_i$ and the *Linker* ensures that for each p, its ϵ-neighborhood is found (by a *Scout* thread) and processed (by the *Linker* thread) once.

Proof: (sketch) Each *Scout* works sequentially on a separate laser (distinct row of the M matrix). Therefore, for each point p, there will be a *Scout* thread that will find p's ϵ-neighborhood exactly once. As the *Linker* interacts for each point p with the assigned *Scout* separately, it is enough to argue about the safety properties of the interaction of the *Linker* with *Scout* i w.r.t. the $list$ infromation exchange: The *Linker* uses the information which is provided by i when it is ready, i.e. through the $list_w^i$ for some w for which $flag_w^i$ is $true$. The linker's resetting of the $flag_w^i = false$ after that, implies that *Scout* i does not overwrite useful information, as i needs to find a workspace w with $flag_w^i = false$ (line 6 in Algorithm 6) in order to write the information about p's ϵ-neighbourhood. The fact that the *Linker*, after $termination_i$ is set for all i (i.e. for all *Scout*s) will iterate once more over all workspaces, implies that it will not miss any updates, since each *Scout* first updates its workspace and then sets $termination_i$ to true.

■

Claim 4. If points p and p' are clustered together at the end of *Lisco*, they are clustered in the same way in *P-Lisco* regardless of the relative ordering of *Scout* thread steps.

Proof: (sketch) (i) If the two points p and p' are in the ϵ-neighbourhood of each other, regardless of which point is processed first, Claim 1 holds, since they will be inserted to the same cluster. (ii) If p and p' are not in the ϵ-neighbourhood of each other but they have to be in the same cluster, there must be a chain of pairs of points between p and p' that are ϵ-neighbors and connect p and p'. By working inductively on the chain of these points, and by using Claim 1 as in the former case here, we argue that each such pair of points will end up in the same cluster, so the whole chain will be in the same cluster regardless of the ordering of the processing of the points. Combining the above with claim 3, we get that the required property holds. ■

C. Parallelization estimation

In the following we provide an outline of the approximate expected behavior of *P-Lisco*.

As explained in § IV-C, the work for each point includes (i) finding its ϵ-neighborhood, and (ii) linking the neighbors. The second part's is additive in latency to the first one in *Lisco*, while it is pipelined (and ideally entirely overlapping with the first, thus "hidden") in *P-Lisco*. Moreover, in *P-Lisco* there is a synchronization cost induced. We expect the speed-up enabled by *P-Lisco* to depend on the parallelization and effect of any redundancy present in the first part, the pipelining (latency-hiding through overlapping) of the second part and the efficiency of synchronization [18].

More specifically, having T denote the number of threads running the *Scout* in parallel, E_s the cost of synchronization and $E_L^{(i)}$ and $E_{PL}^{(i)}$ the expected number of steps of (i) for *Lisco* and *P-Lisco*, respectively, while $E_L^{(ii)}$ and $E_{PL}^{(ii)}$ the corresponding expected cost of linking, we expect *P-Lisco* to perform better than *Lisco* if:

$$E_L^{(i)} + E_L^{(ii)} > max(\frac{E_{PL}^{(i)}}{T}, E_{PL}^{(ii)}) + E_s$$

The least favourable case for *P-Lisco* that the cost for part (ii) is assumed to be additive to the cost of part (i) and the synchronization (i.e. if the linking is not "hidden" in the pipeline); although not realistic, this will give a worst-case estimation for the needed number of *Scout* threads in *P-Lisco* in order for it to process the points faster, i.e.

$$T > \frac{E_{PL}^{(i)}}{E_L^{(ii)} + E_L^{(i)} - E_s}$$

This, too, is a worst-case related calculation, with an even higher over-estimation, since it does not consider the overlap between $E_{PL}^{(ii)}$ and $E_{PL}^{(i)}$, which is largely data-dependent and hence harder to estimate in a data-agnostic way.

VI. Evaluation

We evaluate *Lisco* and *P-Lisco* on varying hardware architectures and compare with the baseline *PCL_E_Cluster*. Since the clustering outcomes of all the algorithms are the same, we focus on the completion time of each approach.

A. Data

We use both real-world and synthetic datasets. The first is from the *Ford Campus* dataset [23] while the second has been generated using the Webots simulator [19]. We use synthetic datasets to explore the effect of data on the algorithms' performance, by designing environments with different number of *reflected points* that have been left after removing the ground points.

We composed five scenarios for synthetic datasets, all presented in Figure 5. In all the environments, a VelodyneHDL64E is used to generate a dataset within one physical rotation. Therefore, the number of steps and lasers in all environments are the same, which results in the same total number of points (including NULL points and ground points), equal to 72000. However, the number of reflected points depends on the scenario. The first four scenarios represent outdoor environments with several vehicles, pedestrians, buildings, etc. In all of them, the LiDAR is located on the black vehicle. Among these scenarios, SCEN1 and SCEN2 have fewer objects than SCEN 3 and SCEN 4, thus influencing the number of merge operations. Moreover, in SCEN1 and SCEN3, objects are placed close to the LiDAR while in SCEN2 and SCEN4, they are placed far from it, thus resulting in varying numbers of reflected beams (i.e. non-NULL data points). As it is shown in the caption of Figure 5, near objects reflect more points (i.e. induce larger n), while far objects reflect fewer points, with a larger gap between two nearby points. Furthermore, SCEN5 shows an indoor environment as an example of production environment with robot arms, assembly line warehouse objects and humans; this scene has an even higher number of reflected points than the others, implying a processing-intensive case. In this environment, the LiDAR is located on the purple column.

For the real-world data, we use the *Ford Campus* dataset, collected by an autonomous ground vehicle testbed, with a Velodyne HDL-64E LiDAR scanner [23]. The vehicle path trajectory in this dataset contains large and small objects (e.g. buildings, vehicles, pedestrians, vegetation). The number of scenes (rotations) is 2880, with minimum number of reflected points 5000, maximum 75550, and average 50225, which is lower than SCEN5 but higher than other scenarios.

B. Evaluation setup

We run PCL's $EuclideanClusterExtraction$ class for *PCL_E_Cluster*, which is implemented in C++. We also implemented *Lisco* and *P-Lisco* in C++ and compiled with gcc-4.8.4 using the -O3 optimization flag. To cover a range of relevant infrastructures, we run the experiments on two different hardware platforms, representative of the possible deployment environments. The first one is Intel(R) Xeon(R), a high-end server and can be used in contexts such as

(a) SCEN1 - The objects are close to the LiDAR with 24549 reflected points

(b) SCEN2 - The objects are far from the LiDAR with 14774 reflected points

(c) SCEN3 - The objects are close to the LiDAR with 38306 reflected points

(d) SCEN4 - The objects are far from the LiDAR with 16361 reflected points

(e) SCEN5 - The high density environment with 56826 reflected points

Figure 5: *Different scenarios for synthetic datasets. In SCEN1-SCEN4, the LiDAR is located on the black car in the middle while in SCEN5, it is located on the purple column.*

factories and similar production environments, where the cost and size of such a device are justified. The second platform is ODROID-XU3, a resource-constrained device to be used at deployments such as in a vehicle or robot. We can note that distributing the cluster-processing would introduce communication latencies that depend highly on the deployments and may need to resort to approximations, such as downscaling, resulting in less precise outcomes. In parallel, as we see later in this section, the benefits of *Lisco* and *P-Lisco* can imply compliance with real-time requirements even on such resource-constrained platforms.

1) Intel(R) Xeon(R): This system has two 2.10 GHz Intel(R) Xeon(R) E5-2695 processors, 64 GB memory and runs Ubuntu 16.04 Linux. Each processor has 18 cores with 32 KB of L1, 256 KB of L2 and 45 MB of L3 cache.

2) ODROID-XU3: The ODROID-XU3 heterogeneous hardkernel [10] features Samsung Exynos5422 SoC with a Cortex-A15 2.0 GHz quad core CPU block and a Cortex-A7 1.4 GHz quad core CPU block. Each core has a 32 KB L1 instruction cache and a 32 KB L1 data cache. The four cores on Cortex-A15 block share a 2 MB L2 cache, while the cores on Cortex-A7 block share a 512 KB L2 cache. The board runs Ubuntu 16.04 Linux.

All the experiments are run with the *minPts* parameter set to 10 and the results (the average execution time over several runs) are with *confidence level* 99%. The *execution time* is measured from the time instant the first data point of the dataset is received until the time instant when the clustering algorithm has processed all data points of one full physical rotation. The results have been collected for different values of ϵ. A higher value of ϵ implies a larger ϵ-neighborhood for a point, hence the clustering algorithm needs more time to search in the neighborhood.

C. Performance evaluation - Intel Xeon

For *P-Lisco* on the Intel platform, we use 1,2,4,8, and 16 *Scout* threads and 10 work-spaces per each, for both datasets.

1) Synthetic datasets: Figure 6 shows the average execution time over 20 runs, for different scenarios. We chose $[0.1 - 1]$ meters for ϵ, in which all objects are correctly identified (ϵ values higher than 1 m or lower than 0.1 m would cluster distinct objects together, or characterize more points as noise, respectively).

As expected, by increasing ϵ, the execution time increases for all the algorithms. However, using more *Scout*s in *P-Lisco*, makes the algorithm less sensitive to the value of ϵ. Furthermore, regardless of the value of ϵ, *Lisco* is always faster than *PCL_E_Cluster*. For instance, *Lisco* is up to 77% faster in SCEN3, and up to 69% faster in SCEN5, compared to *PCL_E_Cluster*. Also, *P-Lisco* with more than 4 threads is faster than *Lisco*. This is because with fewer number of threads in *P-Lisco* the parallelization benefit is balanced by the cost of synchronization (especially in the environments with light workload like SCEN2 and SCEN4, which are more appropriate cases for *Lisco*).

We can observe that in the environments with relatively light workload (e.g. SCEN2 and SCEN4), the performance of *PCL_E_Cluster* is close to *P-Lisco* with only one *Scout*. The reason is that in these environments the number of points is small and the objects are far from each other, hence the kd-tree becomes less of a bottleneck. Nevertheless, in the same environments the latency of *P-Lisco* with higher number of *Scout*s can meet real-time requirements. This means all the clusters are ready by the time the LiDAR collects the last points of the rotation. For instance, in SCEN3, *P-Lisco*16 is up to 93% faster than *PCL_E_Cluster* and up to 79% faster than *Lisco*.

The scalability with respect to the number of reflected points is shown in Figure 7. As it can be noticed, the benefits of *P-Lisco*'s parallelism are even higher for datasets containing a larger amount of reflected points.

2) Real-world dataset: Figure 8 shows the average execution time for ϵ values 0.3, 0.4, and 0.7 over 2280 rotations of the real dataset. Recall that the number of reflected points ranges berween 5000 and 75550, with an average of 50225, which is lower than SCEN5 but higher than the other scenarios. As shown in the figure, *Lisco* outperforms *PCL_E_Cluster* in real-world datasets. Regarding ϵ, we observe a phenomenon similar to the one observed for the synthetic scene: Here too, with increasing ϵ, i.e. higher

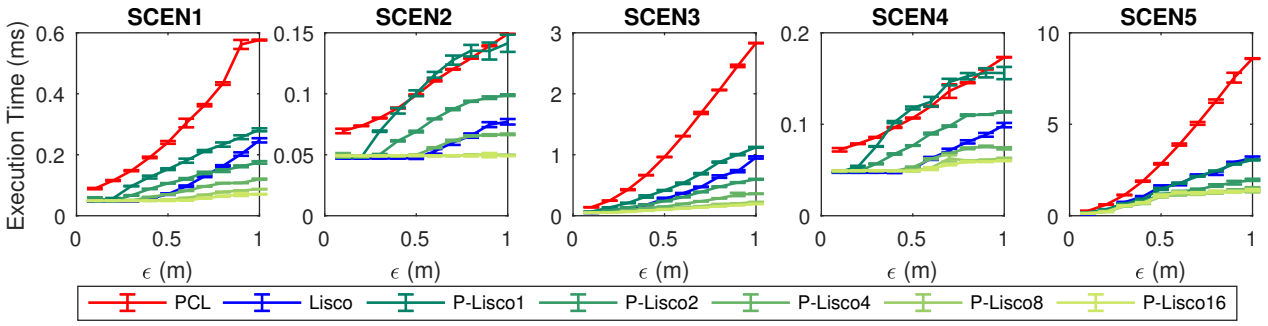


Figure 6: *Average execution time on synthetic datasets with confidence level 99% over 20 runs (minPts = 10, different ϵ values, Intel Xeon)*

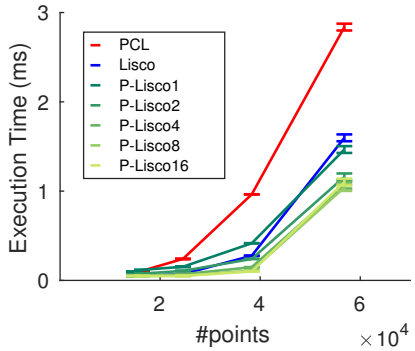


Figure 7: *Scalability of PCL, Lisco, and P-Lisco with respect to the number of points (Intel Xeon)*

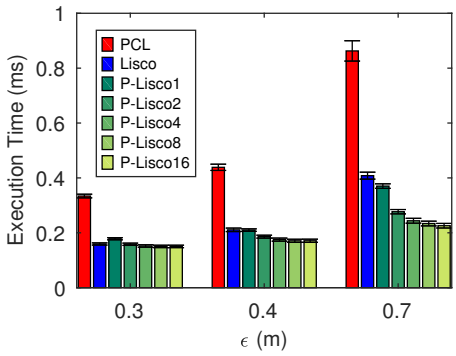


Figure 8: *Average execution time over 2280 rotations from real dataset (minPts = 10, different ϵ values, Intel Xeon)*

workload due to the bigger neighbor mask and higher number of distance computations, the performance of *P-Lisco* with more *Scout*s is significantly better than *Lisco* and *PCL_E_Cluster*. Moreover, similar to the synthetic datasets, *P-Lisco* with higher number of *Scout*s is more stable against the value of ϵ, compared to *PCL_E_Cluster* and *Lisco*.

D. Performance evaluation - ODROID-XU3

On this platform, we run *P-Lisco* with 1,2, and 4 *Scout* threads and 10 work-spaces per *Scout*. As explained in subsubsection VI-B2, different cores in ODROID-XU3 have different characteristics. Hence, we use thread pinning to prevent *Scout*s and *Linker* from being assigned to different cores and have consist results. We conducted experiments on both blocks separately. As the results for the two blocks are similar in the comparative behaviour between the proposed algorithms and the baseline (only differing in the absolute numbers), in the following we show the results of pinning *Scout* threads on the Cortex-A15 block.

1) Synthetic datasets: Figure 9 shows the average execution time over 20 runs of *PCL_E_Cluster* and *Lisco*, for different scenarios. As it can be noticed, the gap between the curves is even bigger, compared to the previous platform. For instance, in SCEN3 *Lisco* is up to 97% faster than *PCL_E_Cluster*. The figure also indicates that *PCL_E_Cluster* is not suitable for embedded devices with limited resources. For example, in SCEN1 with ϵ equal to 1 meter, *PCL_E_Cluster* takes about 10 seconds to process the points while *Lisco* requires less than one second to produce the same results. The difference is largely due to the continuous processing nature of *Lisco* (and *P-Lisco*), resulting in significantly higher data-locality.

To allow to observe the parallelization behaviour from *Lisco* to *P-Lisco*, Figure 10 shows separately the average execution time over 20 runs of *Lisco* (the same experiment as Figure 9) and *P-Lisco* with 1,2, and 4 *Scout* threads. As shown in the figure, *P-Lisco* with 1 *Scout* is slower than *Lisco*, because by using only one *Scout*, the synchronization cost overcomes the parallelization benefit.

Figure 11 shows the execution time with respect to the number of reflected points. *PCL_E_Cluster* can hardly scale (e.g. requiring more than 40 sec to process a "busy" scene), while *Lisco* and *P-Lisco* are more stable against the number of reflected points.

2) Real-world dataset: Figure 12 shows the average execution time for the real-world dataset. Similarly as before, when increasing the workload by means of a higher ϵ, the required number of distance computations increases. In this case we can better appreciate that the performance of *P-Lisco* with more *Scout*s is higher than *Lisco*'s one.

Discussion: As mentioned in § IV-C, the worst-case time complexity of *Lisco* is dominated by the total number of

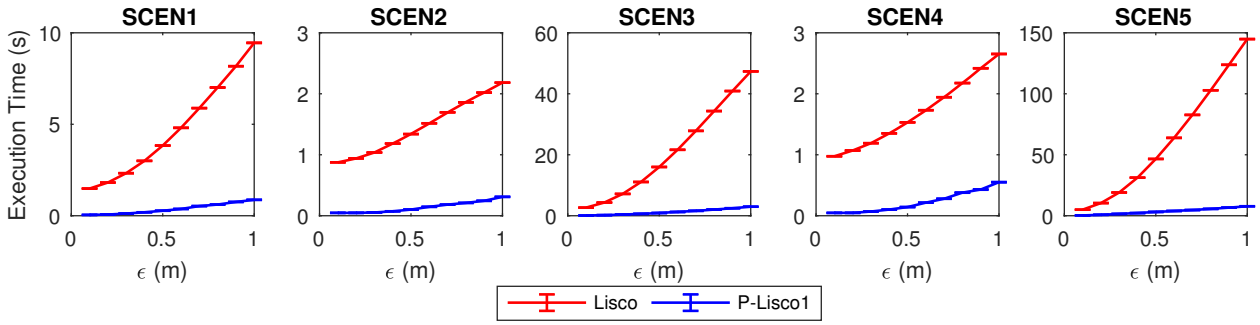


Figure 9: *Average execution time of PCL_E_Cluster and Lisco on synthetic datasets with confidence level 99% over 20 runs (minPts = 10, different ϵ values, ODROID-XU3)*

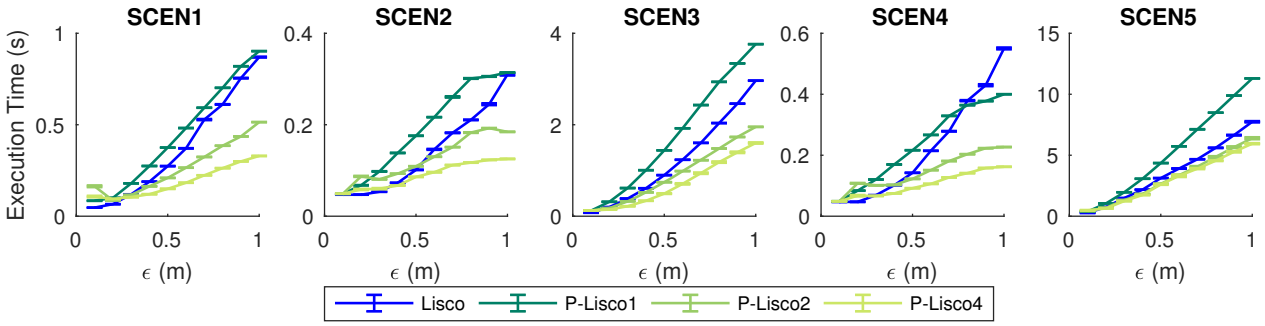


Figure 10: *Average execution time of Lisco and P-Lisco with 1,2, and 4 Scouts on synthetic datasets with confidence level 99% over 20 runs (minPts = 10, different ϵ values, ODROID-XU3)*

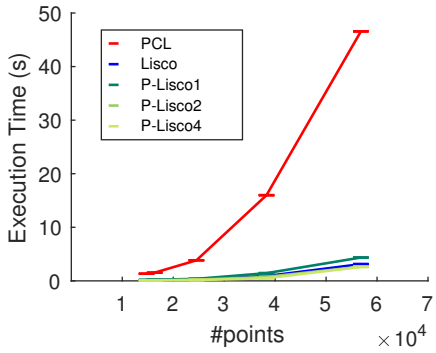


Figure 11: *Scalability of PCL, Lisco, and P-Lisco with respect to the number of points (ODROID-XU3)*

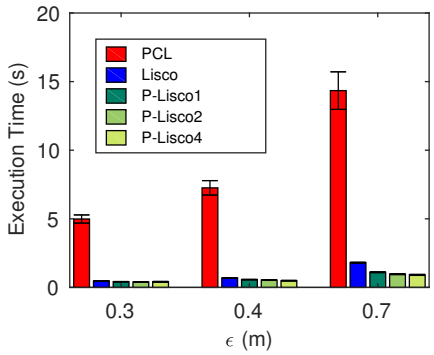


Figure 12: *Average execution time over 2280 rotations from real dataset (minPts = 10, different ϵ values, ODROID-XU3)*

merge-related processing steps (head-changes due to merges, cf. Claim 1), which can be bounded asymptotically by $O(nlogn)$. Figure 13 depicts (i) this worst-case bound as a function of n (more precisely $(nlogn)/2$ to have a clearer visual perception), together with (ii) the corresponding calculation of the empirically observed maximun number of times that points in subclusters have to change their head pointers upon calling `merge`, for each rotation with the corresponding number n of reflected points, based on 2280 rotations of real datasets for *minPts* = 10 and $epsilon = 0.7$, i.e. computation-intensive scenarios. Line n is also shown in the figure for perspective. We can easily observe the gap between the worst-case analytic bound and the actual behaviour based on the large sets of real-world data.

VII. Other Related Work

Data clustering has been studied for several decades and existing algorithms have been categorized into four classes: density-based, partition-based, hierarchy-based, and grid-based [9]. Due to their ability in finding arbitrarily shaped clusters without requiring to know the number of clusters a priori, density-based methods are widely used in different applications. Although working with different semantics compared to *Lisco* and *P-Lisco*, it is useful to note well-

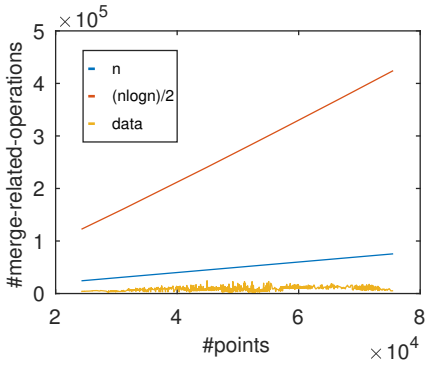


Figure 13: *Asymptotic behaviour of the dominating component of worst case complexity of Lisco and corresponding calculation using 2280 rotations of real datasets for* $\epsilon = 0.7$.

known algorithms of this class, including DBSCAN [4] and OPTICS [2]. Several techniques improve the performance of DBSCAN by parallelizing it [15], [24]. In parallel models, the clustering procedure is divided into three steps: 1) data distribution (e.g. using kd-tree) 2) local clustering (split on several nodes/threads) and 3) merging local clusters. Although the efficiency is improved by splitting the clustering on several nodes, pipelining of steps is not studied; moreover, the execution time is still dominated by the insertion of nodes in a kd-tree, with $O(nlogn)$ expected cost (i.e. not worst-case).

Rusu et al. [26] introduce a Euclidean-distance-based clustering which is a partition-based method for unorganized data points. To find nearest neighbors, a kd-tree is initially built over the dataset and then clustering is performed. In other works [31], [34], an octree is used to identify the neighbors before the clustering procedure starts. A parallel version of Euclidean-distance-based clustering was also added to PCL [27]; this version is designed for GPUs and unlike *Lisco*, has no work-flow pipelining

Since LiDAR data points are implicitly ordered, organizing them (e.g. in a tree) may be avoided, similar to the spirit of this paper. Klasing [14] et al. proposed a clustering method for 2D laser scans that rotate with an independent motor, to cover a 3D environment. While the proposed method compares points across different scans similarly to the problem studied in this work, the semantics of Definition 1 are not enforced, resulting to a lower accuracy than PLC, *Lisco* and *P-Lisco*. Moosmann et al. [20] proposed an approach to turn the scan into an undirected graph to retrieve the neighborhood information of each point during clustering, but they have not studied pipelining in building the graph and the clustering. Zermas et al. [37] recently proposed a clustering method specific to the structure of LiDAR data points. This approach, similarly to previous works, still relies on a kd-tree for some necessary nearest neighbor searches. Moreover, the neighborhood criterion for points clustered in the same scan-line does not take into account the distance of the point from the sensor, thus not guaranteeing Definition 1 semantics. Yin et al. [35] proposed an algorithm based on spherical coordinates to segment LiDAR point-clouds. Their approach clusters together separate objects in the same *azimuth zone* with *small enough* difference in distance from the LiDAR, it does thus not follow the semantics of Definition 1.

There are also parallel point-cloud processing algorithms addressing different types of analysis designed for specific architectures (e.g. GPU [12], FPGA [29]). These approaches focus on processing the data points collected by airbone laser scanners, to construct and update 3D spatial databases.

Clustering LiDAR data points is essential in many applications [13], [16], [28]. Among all, autonomous vehicle ones are among the most challenging since they need fast and accurate results [3], [11], [33], [37]. In [3], a set of voxelisation and meshing segmentation methods are presented. Wang et al. [33] first separate data into foreground and background. Then a clustering procedure is conducted only on the foreground segments, by employing an appropriate clustering method, i.e. *Lisco* and *P-Lisco* can be used here, especially as they point out Euclidean clustering as the suggested one for this type of applications.

VIII. Conclusions and Future Work

This work focuses on a common challenge in big data analysis applications, namely devising efficient methods able to rapidly distill valuable information from raw measurements that come in high-rate streams.

Lisco represents a streaming approach to process LiDAR points while they are being collected. Its parallel counterpart, *P-Lisco*, exploits the parallelism of *Lisco*'s processing pipeline, in an architecture-independent fashion. This characteristic helps to facilitate extraction of clusters in a continuous fashion and contributes to real-time processing. As we show in our evaluation, this is beneficial for high-end server parallel architectures as well as for embedded devices found at the edge of distributed cyber-physical systems.

Follow-up questions include specialized algorithmic implementations on computing architectures such as GPUs and SIMD systems to explore *Lisco*'s and *P-Lisco*'s properties in a broader range of architectures and evaluate its impact on applications that can be deployed on such systems. It will be useful to benefit from efficient fine-grained synchronization methods in streaming-centered and bulk-operations-enabled data structures, as proposed in [6]–[8], [21], [22], [36]. In addition, considering the proposed idea for continuous analysis in problems with similar properties (e.g. temporal locality) is an interesting direction of follow-up work, that can combine with approximations as in [5].

Acknowledgments

Work supported by the Swedish Foundation for Strategic Research, proj. FiC, grant nr. GMT14-0032 and the Swedish Research Council (Vetenskapsrådet) proj. "HARE" grant nr. 2016-03800.

REFERENCES

[1] Annual report, point cloud achievements, profile project, department of production. Technical report, Fraunhofer-Chalmers Research Center for Industrial Mathematics (FCC), 2015. [http://www.fcc.chalmers.se/mediadir/2014/09/fcc_vb_2015.pdf; accessed 18-August-2017].

[2] M. Ankerst, M. M. Breunig, H.-P. Kriegel, and J. Sander. Optics: ordering points to identify the clustering structure. In *ACM Sigmod record*, volume 28, pages 49–60. ACM, 1999.

[3] B. Douillard, J. Underwood, N. Kuntz, V. Vlaskine, A. Quadros, P. Morton, and A. Frenkel. On the segmentation of 3d lidar point clouds. In *Robotics and Automation (ICRA), 2011 IEEE International Conference on*, pages 2798–2805. IEEE, 2011.

[4] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Kdd*, pages 226–231, 1996.

[5] Z. Fu, M. Almgren, O. Landsiedel, and M. Papatriantafilou. Online temporal-spatial analysis for detection of critical events in cyber-physical systems. In *Big Data (Big Data), 2014 IEEE International Conference on*, pages 129–134. IEEE, 2014.

[6] V. Gulisano, Y. Nikolakopoulos, D. Cederman, M. Papatriantafilou, and P. Tsigas. Efficient data streaming multiway aggregation through concurrent algorithmic designs and new abstract data types. *ACM Trans. Parallel Comput.*, 4(2):11:1–11:28, Oct. 2017.

[7] V. Gulisano, Y. Nikolakopoulos, M. Papatriantafilou, and P. Tsigas. Scalejoin: A deterministic, disjoint-parallel and skew-resilient stream join. *IEEE Transactions on Big Data*, 2016.

[8] V. Gulisano, Y. Nikolakopoulos, I. Walulya, M. Papatriantafilou, and P. Tsigas. Deterministic real-time analytics of geospatial data streams through scalegate objects. In *Proceedings of the 9th ACM International Conference on Distributed Event-Based Systems*, DEBS '15, pages 316–317, New York, NY, USA, 2015. ACM.

[9] J. Han, J. Pei, and M. Kamber. *Data mining: concepts and techniques*. Elsevier, 2011.

[10] Hardkernel. Odroid-xu3. http://www.hardkernel.com/main/products/prdt_info.php?g_code=G140448267127.

[11] M. Himmelsbach, F. V. Hundelshausen, and H.-J. Wuensche. Fast segmentation of 3d point clouds for ground vehicles. In *Intelligent Vehicles Symposium (IV), 2010 IEEE*, pages 560–565. IEEE, 2010.

[12] X. Hu, X. Li, and Y. Zhang. Fast filtering of lidar point cloud in urban areas based on scan line segmentation and GPU acceleration. *IEEE Geoscience and Remote Sensing Letters*, 10(2):308–312, 2013.

[13] K. Klasing, D. Wollherr, and M. Buss. A clustering method for efficient segmentation of 3d laser data. In *Robotics and Automation, 2008. ICRA 2008. IEEE International Conference on*, pages 4043–4048. IEEE, 2008.

[14] K. Klasing, D. Wollherr, and M. Buss. Realtime segmentation of range data using continuous nearest neighbors. In *Robotics and Automation, 2009. ICRA'09. IEEE International Conference on*, pages 2431–2436. IEEE, 2009.

[15] S. Kumari, P. Goyal, A. Sood, D. Kumar, S. Balasubramaniam, and N. Goyal. Exact, fast and scalable parallel dbscan for commodity platforms. In *Proceedings of the 18th International Conference on Distributed Computing and Networking*, page 14. ACM, 2017.

[16] W. Li, Q. Guo, M. K. Jakubowski, and M. Kelly. A new method for segmenting individual trees from the lidar point cloud. *Photogrammetric Engineering & Remote Sensing*, 78(1):75–84, 2012.

[17] R. Lindholm and C.-J. Pålsson. *Simultaneous Localization and Mapping*. PhD thesis, Masters thesis, Chalmers University of Technology, 2016.

[18] F. McSherry, M. Isard, and D. G. Murray. Scalability! but at what cost? In *HotOS*. Citeseer, 2015.

[19] O. Michel. Cyberbotics ltd. webots: professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, 1(1):5, 2004.

[20] F. Moosmann, O. Pink, and C. Stiller. Segmentation of 3d lidar data in non-flat urban environments using a local convexity criterion. In *Intelligent Vehicles Symposium, 2009 IEEE*, pages 215–220. IEEE, 2009.

[21] Y. Nikolakopoulos, A. Gidenstam, M. Papatriantafilou, and P. Tsigas. A consistency framework for iteration operations in concurrent data structures. In *Parallel and Distributed Processing Symposium (IPDPS), 2015 IEEE International*, pages 239–248. IEEE, 2015.

[22] Y. Nikolakopoulos, M. Papatriantafilou, P. Brauer, M. Lundqvist, V. Gulisano, and P. Tsigas. Highly concurrent stream synchronization in many-core embedded systems. In *Proceedings of the Third ACM International Workshop on Many-core Embedded Systems*, MES '16, pages 2–9, New York, NY, USA, 2016. ACM.

[23] G. Pandey, J. R. McBride, and R. M. Eustice. Ford campus vision and lidar data set. *The International Journal of Robotics Research*, 30(13):1543–1552, 2011.

[24] M. A. Patwary, D. Palsetia, A. Agrawal, W.-k. Liao, F. Manne, and A. Choudhary. A new scalable parallel dbscan algorithm using the disjoint-set data structure. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, page 62. IEEE Computer Society Press, 2012.

[25] R. B. Rusu. Semantic 3d object maps for everyday manipulation in human living environments. *KI-Künstliche Intelligenz*, 24(4):345–348, 2010.

[26] R. B. Rusu, N. Blodow, Z. C. Marton, and M. Beetz. Close-range scene segmentation and reconstruction of 3d point cloud maps for mobile manipulation in domestic environments. In *Intelligent Robots and Systems, 2009. IROS 2009. IEEE/RSJ International Conference on*, pages 1–6. IEEE, 2009.

[27] R. B. Rusu and S. Cousins. 3d is here: Point cloud library (pcl). In *Robotics and Automation (ICRA), 2011 IEEE International Conference on*, pages 1–4. IEEE, 2011.

[28] A. Sampath and J. Shan. Segmentation and reconstruction of polyhedral building roofs from aerial lidar point clouds. *IEEE Transactions on geoscience and remote sensing*, 48(3):1554–1567, 2010.

[29] K.-T. Shih, A. Balachandran, K. Nagarajan, B. Holland, K. C. Slatton, and A. D. George. Fast real-time lidar processing on FPGAs. In *ERSA*, pages 231–237, 2008.

[30] J. Vilysson. *Robust tracking of dynamic objects in lidar point clouds*. PhD thesis, Masters thesis, Chalmers University of Technology, 2014.

[31] A.-V. Vo, L. Truong-Hong, D. F. Laefer, and M. Bertolotto. Octree-based region growing for point cloud segmentation. *ISPRS Journal of Photogrammetry and Remote Sensing*, 104:88–100, 2015.

[32] I. Wald and V. Havran. On building fast kd-trees for ray tracing, and on doing that in o (n log n). In *Interactive Ray Tracing 2006, IEEE Symposium on*, pages 61–69. IEEE, 2006.

[33] D. Z. Wang, I. Posner, and P. Newman. What could move? finding cars, pedestrians and bicyclists in 3d laser data. In *Robotics and Automation (ICRA), 2012 IEEE International Conference on*, pages 4038–4044. IEEE, 2012.

[34] H. Woo, E. Kang, S. Wang, and K. H. Lee. A new segmentation method for point cloud data. *International Journal of Machine Tools and Manufacture*, 42(2):167–178, 2002.

[35] H. Yin, X. Yang, and C. He. Spherical coordinates based methods of ground extraction and objects segmentation using 3-d lidar sensor. *IEEE Intelligent Transportation Systems Magazine*, 8(1):61–68, 2016.

[36] N. Zacheilas, V. Kalogeraki, Y. Nikolakopoulos, V. Gulisano, M. Papatriantafilou, and P. Tsigas. Maximizing determinism in stream processing under latency constraints. In *Proceedings of the 11th ACM International Conference on Distributed and Event-based Systems*, DEBS '17, pages 112–123, New York, NY, USA, 2017. ACM.

[37] D. Zermas, I. Izzat, and N. Papanikolopoulos. Fast segmentation of 3d point clouds: A paradigm on lidar data for autonomous vehicle applications. In *Robotics and Automation (ICRA), 2017 IEEE International Conference on*, pages 5067–5073. IEEE, 2017.